Power BI Vibes

ChatGPT + GitHub + Power BI, without requiring the user to code

CLIENT GUIDE | v0.1.0

Power BI Vibes is a workflow for building Power BI tools while keeping restricted operational data out of the development conversation when necessary.

You explain the job and the business meaning. ChatGPT handles most repository, Power BI, DAX, TMDL, PBIR and Power Query mechanics.

The basic workflow

1. Create an empty private GitHub repository for your project.
2. Connect GitHub to ChatGPT and give ChatGPT the project repository plus the Power BI Vibes framework repository.
3. Describe what the tool should help you or your team do.
4. Provide only a permitted representation of the source data.
5. ChatGPT builds with synthetic data, commits the PBIP project, and gives you exact local QA steps.
6. Connect the approved real source locally for the production-data smoke test.

Start a project

The client-facing setup should take only a few minutes.

Create an empty private GitHub repository. Then start a ChatGPT conversation and paste the prompt below.

COPY / PASTE INTO CHATGPT

```
I want to build a Power BI tool.

Use this framework:
https://github.com/dudechilling/Power-Bi-Vibes

My private project repository is:
<PASTE YOUR PRIVATE GITHUB REPOSITORY URL>

Read BOOTSTRAP.md in Power-Bi-Vibes and follow it.
Do not ask me to share operational data that I am not permitted to share.
```

What to tell ChatGPT

Describe the job in ordinary language: who uses the tool, what they need to see or decide, what source files exist, how often those files change, and what currently creates friction.

You generally do not need to specify chart types, DAX, TMDL, PBIR, Power Query steps, or Git operations. ChatGPT should ask only when a business definition or tradeoff affects correctness.

What happens next

- ChatGPT inspects and initializes the private repository.
- It records the framework and Microsoft Power BI authoring versions used by the project.
- It asks what the Power BI tool should help you accomplish.
- It creates a report specification, data contract and synthetic development fixture before a large build.

Restricted data

Develop against structure and synthetic fixtures when real rows cannot be shared.

Do not upload a production workbook, CSV or database merely to make development easier. A blank workbook is not automatically sanitized: hidden sheets, names, formulas, connections, metadata and internal terminology can still reveal information.

Safe source options

1. An approved real sample, when your organization permits it.
2. A scrubbed/template file that preserves the useful structure.
3. A metadata-only schema report generated locally.
4. A manually described schema when nothing else can be shared.

LOCAL METADATA INSPECTOR

```
powershell -ExecutionPolicy Bypass -File .\scripts\inspect-source.ps1 `
  -Path "C:\Path\To\Source.xlsx" `
  -OutputPath ".\schema-report.json"
```

Review schema-report.json before sharing it. The inspector omits row values, but schema and terminology can still be sensitive.

Synthetic development data

ChatGPT should generate fictional records that match the permitted schema and exercise real edge cases: every status, nulls, boundary dates, long labels, valid and invalid links, high-cardinality fields and legitimate numeric extremes.

Synthetic QA proves behavior against the fixture. It does not prove production completeness, cleanliness, distribution or performance.

Build, test, and maintain

The repository carries the editable PBIP project and the instructions needed to resume work.

During normal work, the current usable product stays on main. ChatGPT makes small checkpoint commits after validated logical batches. A temporary branch is reserved for substantial experiments on an accepted working product.

Local Power BI testing

For a first local copy:

```
git clone <YOUR-PRIVATE-REPOSITORY-URL> C:\PBI\<PROJECT-NAME>
```

For later updates:

```
cd C:\PBI\<PROJECT-NAME>
git pull --ff-only
```

Open the .pbip file under powerbi/ in Power BI Desktop. ChatGPT should give you a short set of task-based checks with expected results, rather than asking you to "review the dashboard."

Production-data smoke test

- Connect the approved real source only where it is permitted to exist.
- Refresh and verify schema compatibility, relationships, totals, filters and performance.
- Keep restricted screenshots, records and logs local. Return sanitized errors or safe screenshots when needed.
- If the export schema changes later, give ChatGPT a safe updated schema/template so it can update the source adapter without rebuilding the whole report.

Done means

The project validates, requested data-bound functions exist, the rendered layout has been checked, interactions work, acceptance steps are current, and the separate production-data smoke test has passed.